

Mocking using Moq - Implementation Document

Introduction

This exercise explains how to perform unit testing using Moq, NUnit and Dependency Injection. External dependencies are replaced with mock objects so tests run without real services.

Task 1 - Mock Mail Object

- Step 1: Create Class Library CustomerCommLib.
- Step 2: Create IMailSender interface.
- Step 3: Implement MailSender.
- Step 4: Create CustomerComm class and inject IMailSender through constructor.
- Step 5: Build project.
- Step 6: Create CustomerComm.Tests.
- Step 7: Install NUnit, NUnit Test Adapter and Moq.
- Step 8: Mock IMailSender to always return true.
- Step 9: Run the test and verify SendMailToCustomer() returns true.

Task 2 - Mock File Object

- Step 1: Create MagicFilesLib.
- Step 2: Create IDirectoryExplorer interface.
- Step 3: Implement DirectoryExplorer.
- Step 4: Create DirectoryExplorer.Tests.
- Step 5: Mock GetFiles() to return two file names.
- Step 6: Verify collection is not null, count is 2 and contains file.txt.

Task 3 - Mock Database

- Step 1: Create PlayersManagerLib.
- Step 2: Create IPlayerMapper and PlayerMapper.
- Step 3: Create Player class.
- Step 4: Create PlayerManager.Tests.
- Step 5: Mock IsPlayerNameExistsInDb() to return false.
- Step 6: Register player and verify player properties.

Conclusion

Moq and Dependency Injection make unit testing easier by avoiding real SMTP servers, file systems and databases.